

DAA ASSIGNMENT 2

DATA STRUCTURES

Implementation, Complexity Analysis, Testing and Benchmarking

Student: Temujin Gonchar

Group: SE 2518

1. Introduction

This assignment implements three fundamental data structures in Java 17: a dynamic array, a singly linked list, and a minimum binary heap. The purpose is to examine their behavior through unit tests, operation counters, theoretical complexity analysis, and benchmark workloads.

The structures are implemented from scratch. Java collection implementations are not used as the internal storage of the data structures. Maven is used to build the project and JUnit tests are used to check functional behavior.

2. Implementation Summary

2.1 DynamicArray

DynamicArray stores integer values in a resizable contiguous array. Indexed access is direct. When capacity is exhausted, the backing array grows and existing elements are copied. Insertion and removal at an arbitrary index shift elements to preserve order.

2.2 MyLinkedList

MyLinkedList stores values in linked nodes. Access by index and search require traversal. Insertion or removal at the head changes a small number of references, while operations at an arbitrary index first traverse to the relevant position. The current append implementation traverses to the end if it does not maintain a tail reference.

2.3 MinHeap

MinHeap is an array-backed binary min-heap. The minimum element is held at the root. Insertion restores heap order by sifting upward; extraction replaces the root with the last element and sifts downward. The implementation rejects access or extraction from an empty heap.

3. Complexity Analysis

Structure	Operation	Time complexity	Reason
DynamicArray	add(value)	$O(1)$ amortized; $O(n)$ worst	Occasional capacity growth copies elements
DynamicArray	add(index, value)	$O(n)$	Elements may shift right
DynamicArray	remove(index)	$O(n)$	Elements may shift left
DynamicArray	get(index)	$O(1)$	Direct indexed access
DynamicArray	contains(value)	$O(n)$	Linear scan
MyLinkedList	add(value)	$O(n)$ in current design	Traverses to the final node without tail
MyLinkedList	add(index, value)	$O(n)$	Traverses to insertion position
MyLinkedList	remove(index)	$O(n)$	Traverses to

			target/predecessor
MyLinkedList	get(index)	$O(n)$	Sequential traversal
MyLinkedList	contains(value)	$O(n)$	Linear scan
MinHeap	insert(value)	$O(\log n)$	Sift-up along heap height
MinHeap	peekMin()	$O(1)$	Minimum is at root
MinHeap	extractMin()	$O(\log n)$	Sift-down along heap height

Auxiliary storage is $O(n)$ for each structure. DynamicArray and MinHeap use contiguous backing arrays; MyLinkedList uses node objects and references.

4. Benchmark Methodology

The benchmark runner evaluates input sizes $n = 100, 1,000, 10,000$, and $100,000$. Each workload is run five times: one initial warm-up is discarded, and the reported time is the median of the four measured runs (the average of the two central observations after sorting four measurements). The runner writes results to results/results.csv.

Workload	Description	Structures
W1	10,000 indexed reads using reproducible random indices	DynamicArray, MyLinkedList
W2	1,000 contains queries: 500 selected from input data and 500 absent values	DynamicArray, MyLinkedList
W3 Head	1,000 insertions and 1,000 removals at index 0	DynamicArray, MyLinkedList
W3 Middle	1,000 insertions and 1,000 removals at the initial middle index	DynamicArray, MyLinkedList
W4	Insert n values, then extract n minimums; verify nondecreasing output	MinHeap

The CSV records elapsed milliseconds and operation counters (steps, moves, comparisons). Counter definitions are implementation-specific: they reflect the increments made by the instrumented methods and should be interpreted as comparative counts rather than processor instructions.

5. Test Results

The reported Maven test run completed successfully: 30 tests total, with 0 failures, 0 errors, and 0 skipped. DynamicArrayTest contained 11 tests, MyLinkedListTest contained 11 tests, and MinHeapTest contained 8 tests.

The test suite validates core operations and error behavior represented by the current tests. The command used was: mvn clean test.

6. Benchmark Observations

For W1 at $n = 100,000$, the recorded median was 2.175 ms for DynamicArray and 9,013.138 ms for MyLinkedList. This is consistent with direct array indexing versus sequential linked-list traversal.

For W2 at $n = 100,000$, the recorded times were 26.116 ms for DynamicArray and 8,437.820 ms for MyLinkedList. Both contains operations are linear in the worst case; the measured difference can reflect memory locality, node traversal, and runtime effects.

For W3 at $n = 100,000$, head operations took 21.862 ms for DynamicArray and 7,861.828 ms for MyLinkedList. Middle operations took 10.918 ms and 7,410.847 ms, respectively. The linked list avoids shifting an array after

insertion/removal, but locating a middle node requires traversal. The measured results therefore depend on both locating the position and changing the structure.

For W4 at $n = 100,000$, MinHeap completed the insertion-and-extraction workload in 16.729 ms. The heap's expected total time for n insertions and n extractions is $O(n \log n)$.

These are measurements from one reported run of the benchmark program, not universal performance guarantees. Timing can vary with the machine, JVM warm-up, background activity, and implementation details.

7. Loop Invariant Proofs

7.1 Dynamic Array right-shift loop

Consider insertion at index i . The loop copies elements from the end toward i , shifting each one position to the right.

Invariant: Before each iteration at position k , all original elements at indices greater than k and less than the old size have already been copied to their destinations one position to the right; their original values remain available until copied.

Initialization: At the first iteration, k is the last occupied index. No elements have yet been shifted, so the statement holds vacuously for the already-processed suffix.

Maintenance: The assignment `backing[k + 1] = backing[k]` places the current element in its required destination. Decrementing k extends the processed suffix by one element, preserving the invariant.

Termination: When k reaches $i - 1$, every original element from i through the old last index has moved right by one. Index i is free for the new value, and relative order is preserved.

7.2 MinHeap sift-down loop

During sift-down, the element at a current index is compared with its children, and it is swapped with the smaller child when heap order is violated.

Invariant: At the start of each iteration, the subtrees below the current node satisfy the min-heap property, and any possible violation is confined to the current node and its path toward the root of the considered subtree.

Initialization: At the starting position, the children are roots of subtrees that already satisfy the heap property; after root replacement, the only possible violation is at the root.

Maintenance: If the current node is no greater than its children, the heap property holds and the loop stops. Otherwise, swapping with the smaller child places the smaller value above both children; the possible violation moves down to the child position, while the completed portion remains valid.

Termination: The loop stops when the current node is no greater than its children or reaches a leaf. Thus every parent is no greater than its children and the min-heap property is restored.

8. Limitations and Future Improvements

The current linked-list append operation can be improved by maintaining a tail pointer, which would make append $O(1)$ while requiring careful updates after removing the final node. Benchmark counters should be documented precisely per operation, and repeated benchmark sessions on an otherwise idle machine would help assess variability. Additional randomized differential tests against a trusted reference model could strengthen correctness validation.

9. Conclusion

The assignment implements and tests a dynamic array, singly linked list, and min-heap, and measures their behavior across multiple input sizes. The observations broadly illustrate the role of access patterns and asymptotic complexity: arrays support constant-time indexed access, linked lists require traversal for indexed operations, and a binary min-heap supports logarithmic insertion and extraction. The test suite passed, and benchmark output and plots are available as project artifacts.

Appendix: Reproducibility

Run tests: `mvn clean test`

Run benchmark: `mvn exec:java` with the `BenchmarkRunner` main class and compile classpath scope.

Generate plots: `py scripts/plot_results.py`

Benchmark CSV: `results/results.csv`

Plots: `results/plots/`